

Argo CD :

Definition: GitOps is a methodology for managing software infrastructure and deployments using Git as the single source of truth.

Core Concepts:

- **Git as the Source of Truth:** All configuration files, including deployments, services, secrets, etc., are stored in a Git repository.
- **Automated Processes:** Tools like **Argo CD** and Flux detect changes in the Git repository and automatically apply them to the Kubernetes cluster. This ensures that the live infrastructure aligns with the desired state defined in Git.
- **Continuous Deployment:** Any changes made in the Git repository are automatically reflected in the Kubernetes cluster, enabling continuous deployment.

Before Argo CD (Manual Deployment):

- Applications were deployed manually using tools like kubectl, Helm, etc.
- For Kubernetes clusters (e.g., KOPS), configurations like RBAC had to be manually managed. For EKS, IAM credentials were required.
- Challenges:
 - No graphical user interface (GUI) to track deployment status.
 - Increased risk of human error and security vulnerabilities.
 - Reliance on third-party tools.

Advantages of Using Argo CD:

1. Automated State Reconciliation:

- Argo CD ensures that the live cluster state matches the desired state defined in Git.
- Manual changes made directly to the cluster using kubectl are rejected, ensuring consistency.

2. Change Tracking:

- Changes made in Git (e.g., increasing replicas) are automatically applied to the cluster.
- Maintains a detailed history of changes for audit and review.

3. Rollback Capability:

- Easy rollback to previous configurations using Git history if issues arise.

4. Disaster Recovery:

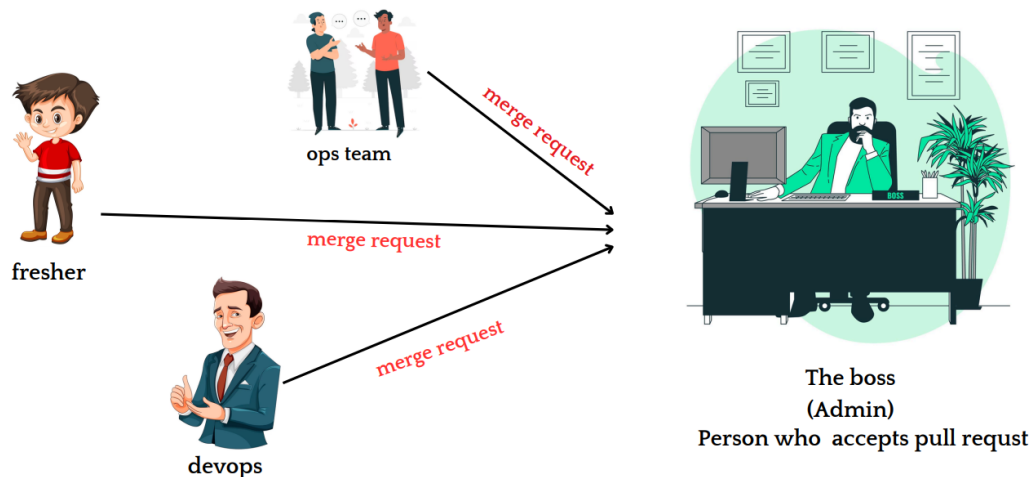
- If the cluster is deleted or affected by network issues, configurations stored in Git can be reapplied to restore the cluster.

Conclusion:

Argo CD simplifies Kubernetes application deployments by automating synchronization with Git, providing robust disaster recovery, and ensuring consistency and visibility across clusters. By adopting GitOps principles with Argo CD, teams can achieve greater efficiency, security, and reliability in their DevOps workflows.

Access controls:

1. In K8s, we use RBAC concept to give cluster permissions which is high configurations and high maintenance. But in ArgoCD, just look over here



CREATE CLUSTER USING KOPS :

INSTALL HELM:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
```

```
chmod 700 get_helm.sh
```

```
./get_helm.sh
```

```
helm version
```

INSTALL ARGO CD USING HELM :

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

```
kubectl get all -n argocd
```

```
[root@ip-172-31-81-215 ~]# kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
i-099c26676c78e232d               Ready    node     39s   v1.24.17
i-0f17clefcc3adde19               Ready    node     41s   v1.24.17
i-0fcc2d56090f7056d               Ready    control-plane 101s   v1.24.17
[root@ip-172-31-81-215 ~]# curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
[WARNING] Could not find git. It is required for plugin installation.
Downloading https://get.helm.sh/helm-v3.16.4-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
version.BuildInfo{Version:"v3.16.4", GitCommit:"7877b45b63f95635153b29a42c0c2f4273ec45ca", GitTreeState:"clean", GoVersion:"go1.22.7"}
[root@ip-172-31-81-215 ~]#
[root@ip-172-31-81-215 ~]# kubectl create ns argocd
namespace/argocd created
[root@ip-172-31-81-215 ~]# kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
customresourcedefinition.apiextensions.k8s.io/applications.argoproj.io created
customresourcedefinition.apiextensions.k8s.io/applicationsets.argoproj.io created
customresourcedefinition.apiextensions.k8s.io/appprojects.argoproj.io created
serviceaccount/argocd-application-controller created
serviceaccount/argocd-applicationset-controller created
serviceaccount/argocd-dex-server created
```

```
[root@ip-172-31-81-215 ~]# kubectl get all -n argocd
```

NAME	READY	STATUS	RESTARTS	AGE
pod/argocd-application-controller-0	1/1	Running	0	79s
pod/argocd-applicationset-controller-654587b9d-c99qj	1/1	Running	0	80s
pod/argocd-dex-server-5d5884b54d-4sfjb	1/1	Running	0	80s
pod/argocd-notifications-controller-59d445bb95-xfc2k	1/1	Running	0	80s
pod/argocd-redis-7f68f49b74-v8cg2	1/1	Running	0	80s
pod/argocd-repo-server-5bcd895d7-5k9v9	1/1	Running	0	80s
pod/argocd-server-7bdcb948c4-p9v1m	1/1	Running	0	80s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/argocd-applicationset-controller	ClusterIP	100.65.182.109	<none>	7000/TCP,8080/TCP	81s
service/argocd-dex-server	ClusterIP	100.71.193.128	<none>	5556/TCP,5557/TCP,5558/TCP	81s
service/argocd-metrics	ClusterIP	100.71.49.53	<none>	8082/TCP	80s
service/argocd-notifications-controller-metrics	ClusterIP	100.68.36.108	<none>	9001/TCP	80s
service/argocd-redis	ClusterIP	100.66.135.74	<none>	6379/TCP	80s
service/argocd-repo-server	ClusterIP	100.70.211.222	<none>	8081/TCP,8084/TCP	80s
service/argocd-server	ClusterIP	100.66.237.255	<none>	80/TCP,443/TCP	80s
service/argocd-server-metrics	ClusterIP	100.67.124.76	<none>	8083/TCP	80s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/argocd-applicationset-controller	1/1	1	1	80s
deployment.apps/argocd-dex-server	1/1	1	1	80s
deployment.apps/argocd-notifications-controller	1/1	1	1	80s
deployment.apps/argocd-redis	1/1	1	1	80s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/argocd-applicationset-controller-654587b9d	1	1	1	80s
replicaset.apps/argocd-dex-server-5d5884b54d	1	1	1	80s
replicaset.apps/argocd-notifications-controller-59d445bb95	1	1	1	80s
replicaset.apps/argocd-redis-7f68f49b74	1	1	1	80s
replicaset.apps/argocd-repo-server-5bcd895d7	1	1	1	80s
replicaset.apps/argocd-server-7bdcb948c4	1	1	1	80s

NAME	READY	AGE
statefulset.apps/argocd-application-controller	1/1	80s

```
[root@ip-172-31-81-215 ~]#
```

EXPOSE ARGOCD SERVER:

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
```

```
yum install jq -y
```

```
export ARGOCD_SERVER='kubectl get svc argocd-server -n argocd -o json | jq --raw-output  
'status.loadBalancer.ingress[0].hostname''
```

```
echo $ARGOCD_SERVER
```

The above command will provide load balancer URL to access ARGO CD

```
[root@ip-172-31-81-215 ~]# kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
service/argocd-server patched (no change)
[root@ip-172-31-81-215 ~]# yum install jq -y
Last metadata expiration check: 0:36:56 ago on Thu Dec 19 06:00:00 2024.
Package jq-1.7.1-48.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-81-215 ~]# export ARGOCD_SERVER='kubectl get svc argocd-server -n argocd -o json | jq --raw-output 'status.loadBalancer.ingress[0].hostname''
[root@ip-172-31-81-215 ~]# echo $ARGOCD_SERVER
kubectl get svc argocd-server -n argocd -o json | jq --raw-output status.loadBalancer.ingress[0].hostname
[root@ip-172-31-81-215 ~]# kubectl get svc argocd-server -n argocd -o json | jq --raw-output status.loadBalancer.ingress[0].hostname
a073573b226f14bfa9e29827f35f28f9-1034855171.us-east-1.elb.amazonaws.com
[root@ip-172-31-81-215 ~]#
```

```
[root@ip-172-31-81-215 ~]# kubectl get svc -n argocd
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
argocd-applicationset-controller	ClusterIP	100.65.182.109	<none>	TCP, 8080/TCP	8m24s
argocd-dex-server	ClusterIP	100.71.193.128	<none>	TCP, 5557/TCP, 5558/TCP	8m24s
argocd-metrics	ClusterIP	100.71.49.53	<none>	TCP	8m23s
argocd-notifications-controller-metrics	ClusterIP	100.68.36.108	<none>	TCP	8m23s
argocd-redis	ClusterIP	100.66.135.74	<none>	TCP	8m23s
argocd-repo-server	ClusterIP	100.70.211.222	<none>	TCP, 8084/TCP	8m23s
argocd-server	LoadBalancer	100.66.237.255	a073573b226f14bfa9e29827f35f28f9-1034855171.us-east-1.elb.amazonaws.com	698/TCP, 443:32320/TCP	8m23s
argocd-server-metrics	ClusterIP	100.67.124.76	<none>	TCP	8m23s

```
[root@ip-172-31-81-215 ~]#
```

TO GET ARGO CD PASSWORD:

```
export ARGO_PWD='kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d'
```

```
echo $ARGO_PWD
```

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

```
[root@ip-172-31-81-215 ~]# export ARGO_PWD='kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d'
echo $ARGO_PWD
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
866p1coAAFF0M6cV[root@ip-172-31-81-215 ~]#
```

To access the Argo CD GUI, follow these steps:

1. **Retrieve the LoadBalancer URL:** Use the following command to get the LoadBalancer URL of the Argo CD Service. This URL will allow you to access the Argo CD web interface:
2. `kubectl get svc -n argocd`

Look for the service of type LoadBalancer. The external IP or hostname listed will be your Argo CD URL.

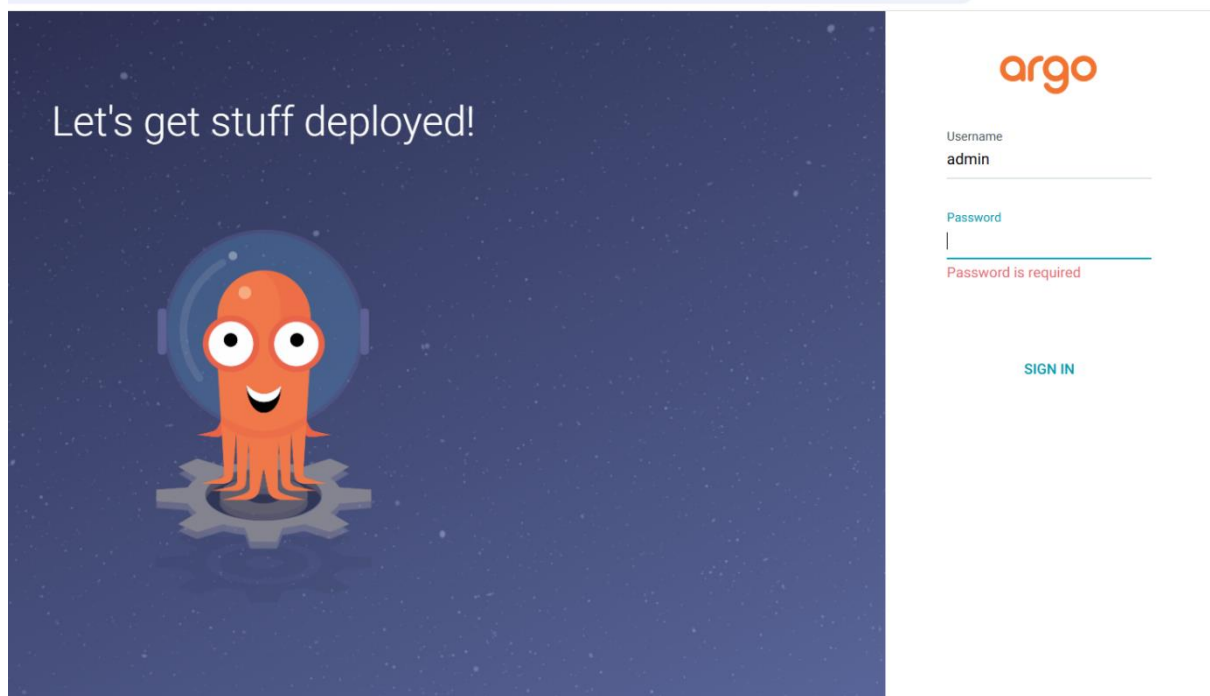
3. Default Username and Password:

- The default username for Argo CD is admin.
 - To retrieve the default password, run the following command:
4. `kubectl get secrets -n argocd argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d`

This command extracts and decodes the password stored in the argocd-initial-admin-secret secret.

5. Login to the Argo CD GUI:

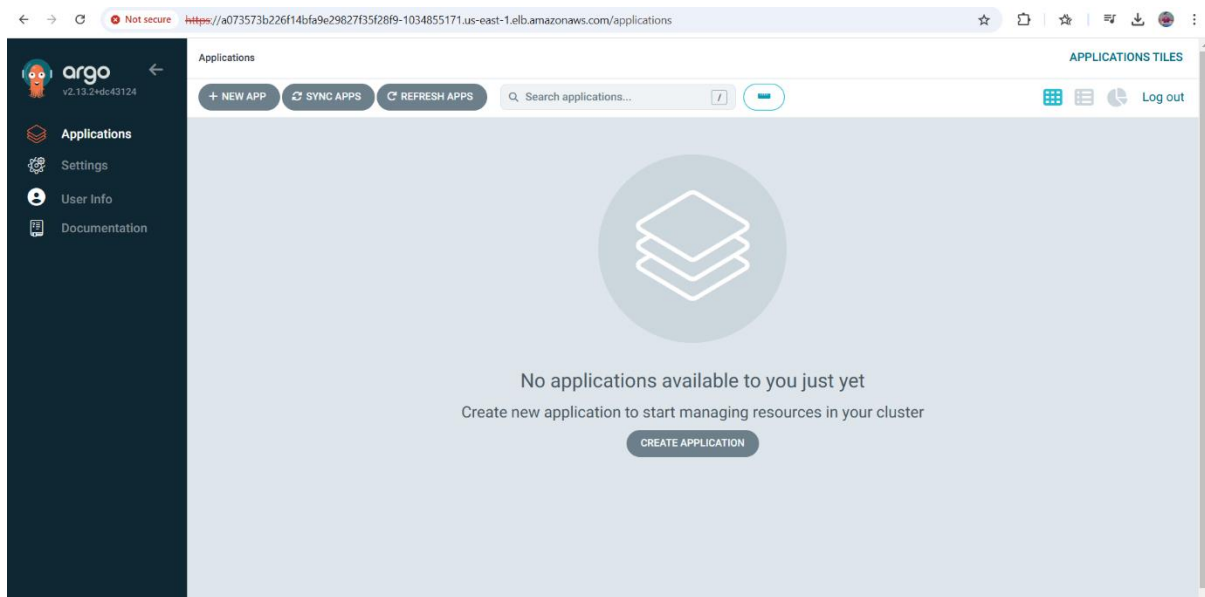
- Open the LoadBalancer URL in your browser.
- Use the username admin and the retrieved password to log in to the Argo CD GUI.



Creating vim deployment.yml file which includes deployment and service files:

```
1  ---
2  apiVersion: apps/v1
3  kind: Deployment
4  metadata:
5    name: my-app
6  spec:
7    replicas: 3
8    selector:
9      matchLabels:
10       app: my-app
11    template: # Corrected from "templates" to "template"
12      metadata:
13        labels:
14          app: my-app
15      spec:
16        containers:
17        - name: cont-1
18          image: shaikmustafa/dm
19          ports:
20            - containerPort: 80
```

```
22  ---
23  apiVersion: v1
24  kind: Service
25  metadata:
26    name: devops
27  spec:
28    type: NodePort
29    selector:
30      app: my-app
31    ports:
32      - port: 8080
33        targetPort: 80
34        nodePort: 30001 # Removed invalid comment
35
36
```



To create a new application in ArgoCD, follow these steps:

1. **Application Name:** Set the application name as swiggy.
2. **Project Name:** Select the project as default.

Source Settings:

- **Repository URL:** Provide the URL of the GitHub repository that contains the deployment.yml and service.yml files.
- **Revision:** Set the revision to HEAD.
- **Path:** Specify the path as the current directory in the GitHub repository by setting it to ./.

SOURCE

Repository URL

`https://github.com/Reddi-Jagadeesh/k8s-repo.git` GIT ▼

Revision

HEAD Branches ▼ ?

Path

`./`

Destination Settings:

- **Cluster URL:** Use the default cluster URL provided by ArgoCD.
- **Namespace:** Manually create a namespace named sample and set it as the target namespace.

DESTINATION

Cluster URL

`https://kubernetes.default.svc` URL ▼

Namespace

`sample`

Final Step: Click **Create** to complete the application setup.

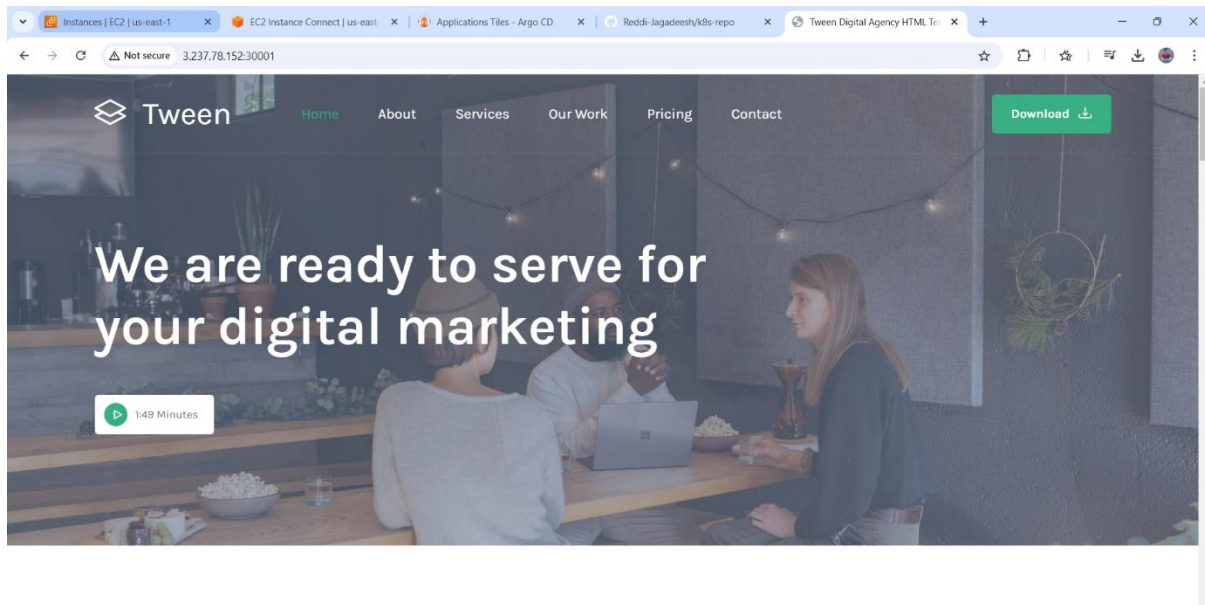
This will configure the `swiggy` application to deploy resources based on the manifests (`deployment.yml` and `service.yml`) in the specified GitHub repository.

The screenshot shows the ArgoCD web interface. On the left is a sidebar with the Argo logo and navigation links: Applications, Settings, User Info, and Documentation. Below these are sections for 'Favorites Only', 'SYNC STATUS' (Unknown: 0, Synced: 1, OutOfSync: 0), and 'HEALTH STATUS' (Unknown: 0). The main panel is titled 'Applications' and contains buttons for '+ NEW APP', 'SYNC APPS', and 'REFRESH APPS', along with a search bar. A modal window displays the configuration for the 'swiggy' application:

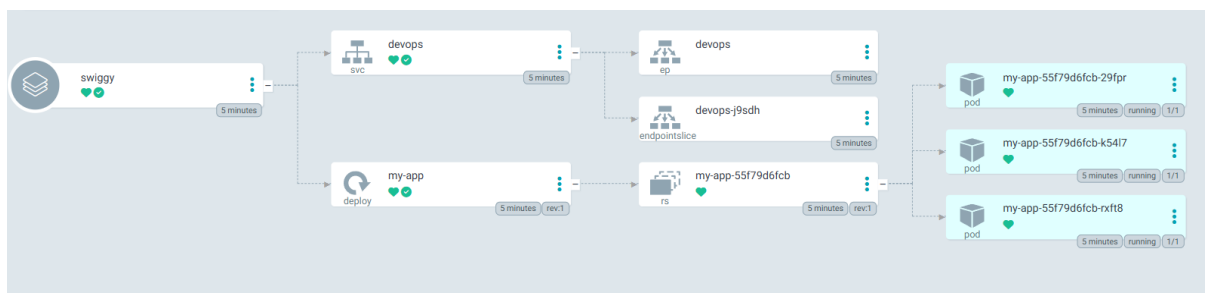
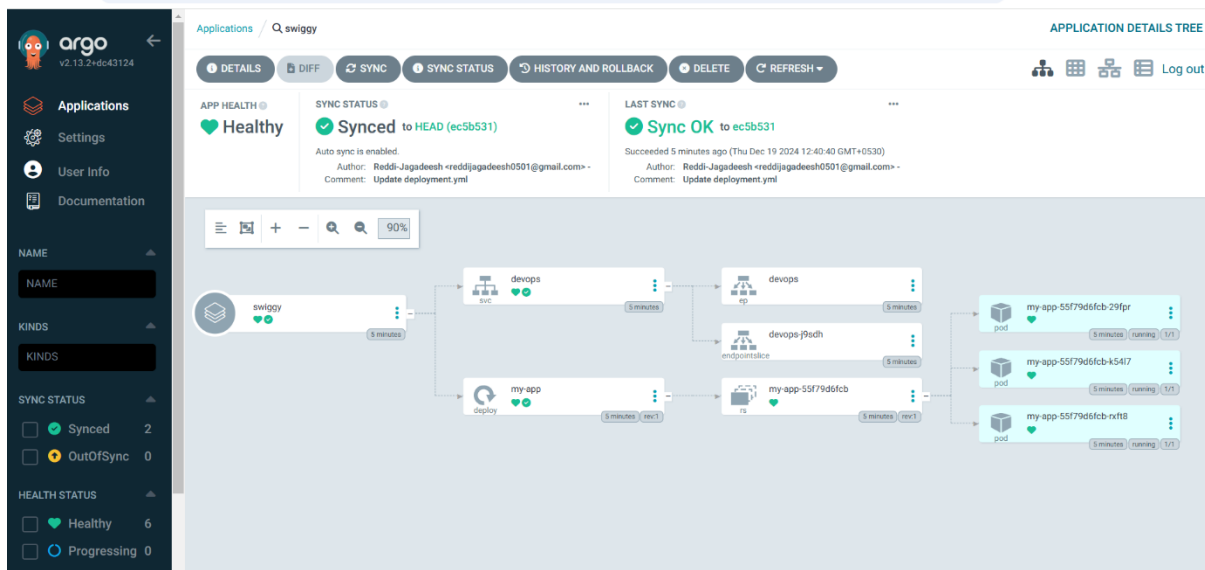
- Project:** default
- Labels:**
- Status:** Healthy (green heart icon) Synced (green checkmark icon)
- Repository:** https://github.com/Reddi-Jagadeesh/k8...
- Target R...** HEAD
- Path:** ./
- Destinati...** in-cluster
- Namesp...** sample
- Created ...** 12/19/2024 12:40:39 (a few seconds a...)
- Last Sync:** 12/19/2024 12:40:40 (a few seconds a...)

At the bottom of the modal are three buttons: SYNC, REFRESH, and DELETE.

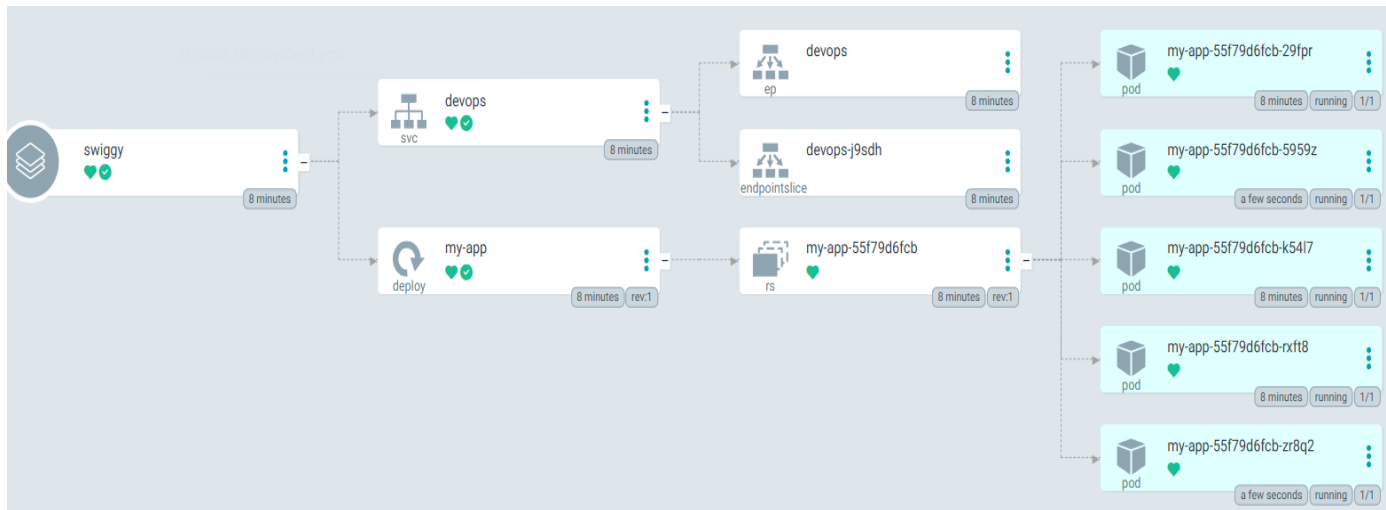
Now we can access the application by using the loadbalancer or ip address :



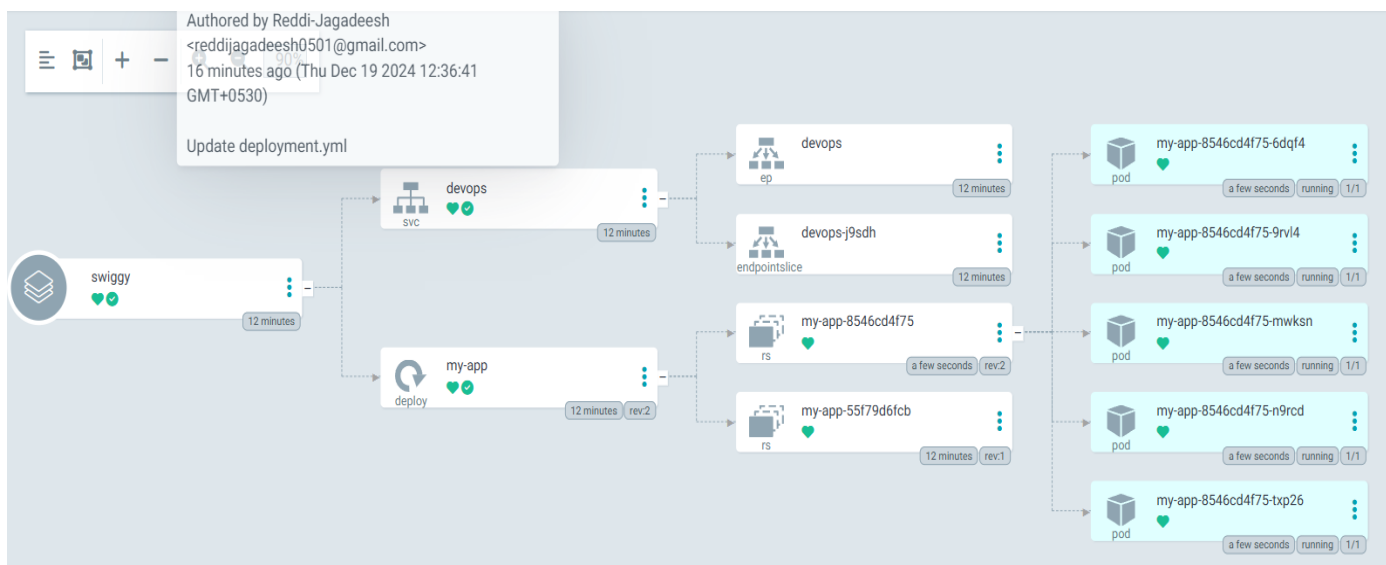
Visualization of creating the pods in argocd:



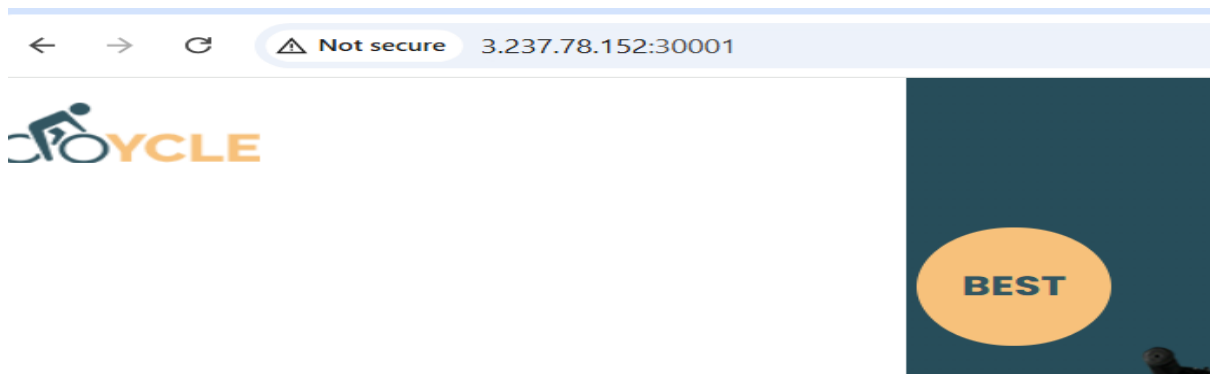
1. After scaling the replicas from 3 to 5 in the deployment.yml file and committing the changes to the GitHub repository, the changes are automatically reflected in ArgoCD. By refreshing the application in ArgoCD, you can observe that the number of pods has increased from 3 to 5.
2. This demonstrates how ArgoCD enables seamless automation for scaling pods, allowing you to easily upscale or downscale replicas by simply updating and committing changes to the GitHub repository.



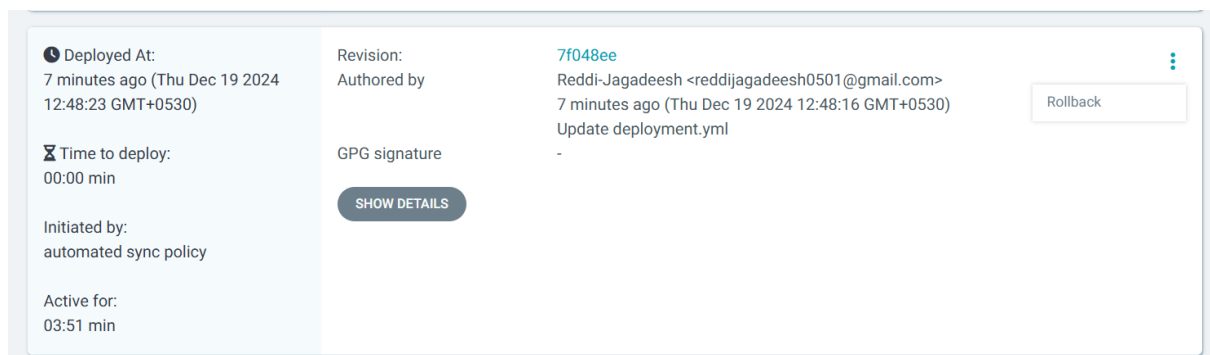
1. After updating the image in the existing deployment.yml file and committing the changes to the GitHub repository, the changes are automatically reflected in ArgoCD.
2. ArgoCD visualizes the new image being deployed. The old pods running the previous image are terminated, and new pods are created to run the updated image. This process demonstrates how ArgoCD ensures automated and seamless updates for containerized applications by synchronizing changes from the Git repository.



New image :



Rolling back to previous version :



This is the first image we deployed, but we have now rolled it back to the previous version using the cycle image :

